CURS:	DAM2	**DATA:**	Setembre-2026	**Qualificació**	
MÒDUL:	M0490 - Programació Serveis i Processos				
TEMA:	Enciptació i desencriptació de dades				
RA	RA5				
Professors	Cristina Reina - Carles Bonet				

Enunciat:

Formeu part de l'equip de desenvolupament d'una empresa que necessita **enviar petits missatges protegits entre diferents aplicacions**.

L'empresa no vol que el programa principal conegui els detalls de com es protegeixen els missatges. Per aquest motiu, **us demana crear una classe criptogràfica independent i reutilitzable**.

La classe haurà de permetre:

MISSATGE + CLAU → Procés d'ENCRIPTACIÓ → MISSATGE ENCRIPTAT

MISSATGE ENCRIPTAT + CLAU → Procés de DESENCRIPTACIÓ → MISSATGE ORIGINAL

La vostra missió és **dissenyar el vostre propi sistema de xifrat** i implementar-lo com una classe reutilitzable.

Un exemple per entendre com crear el vostre propi sistema:

Suposem que tenim:

- Missatge: HOLA
- Clau: 3

Podem utilitzar la clau per desplaçar cada lletra tres posicions:

- H → K
- O → R
- L → O
- A → D

Per tant: HOLA → KROD

Per recuperar el missatge original, fem el procés invers: KROD → HOLA.

1. **Investiga el funcionament**

   Respon aquestes preguntes:

   · **Què passa si encriptem dues vegades el mateix missatge amb la mateixa clau?**

   El resultat és sempre el mateix, perquè amb el mateix missatge i la mateixa clau s'apliquen exactament les mateixes operacions. Per exemple, si encriptem HOLA amb la clau 3 mitjançant un sistema de desplaçament, obtenim KROD. Si ho repetim amb el mateix missatge i la mateixa clau, torna a sortir KROD.

   · **Què passa si canviem la clau? Prova HOLA + clau 3 i HOLA + clau 7.**

   El resultat canvia, perquè cada clau desplaça les lletres un nombre diferent de posicions.

   - HOLA amb la clau 3 → KROD
   - HOLA amb la clau 7 → OVSH

   · **Què passa si intentem desencriptar utilitzant una clau incorrecta?**

   No recuperem el missatge original. Per exemple, si desencriptem KROD amb la clau 3, obtenim HOLA. En canvi, si fem servir la clau 2, obtenim IPMB. Per tant, necessitem utilitzar la mateixa clau amb què s'ha encriptat el missatge per poder recuperar correctament el text original.

   Desencripta(Encripta("HOLA", "123"), "123") = "HOLA"

2. **Dissenya l'algoritme propi d'encriptació i desencriptació**

   Abans de programar, heu de dissenyar el vostre sistema de xifrat.

   Podeu utilitzar, per exemple:

   · un sistema de substitució.

   · un sistema de desplaçament.

   · una combinació de diferents operacions.

   La clau ha de participar obligatòriament en el procés.

Nom del vostre sistema	Xifrat de 3 fases
Tipus de sistema	☐ **Substitució** ☐ **Desplaçament** ☐ **Combinació** ☐ **Altres:** Combinació de desplaçament, transposició i substitució.
Tipus de clau	Clau de text
Com funciona l'encriptació?	El sistema utilitza tres fases diferents.

	En la primera fase, cada lletra de l'alfabet de la A a la Z es converteix en un valor numèric entre 0 i 25. Els espais, accents, números i altres símbols es mantenen sense modificar. A continuació, al valor de cada caràcter del missatge se li suma el valor del caràcter corresponent de la clau i també la seva posició dins del missatge. Si el resultat supera el nombre de caràcters del nostre alfabet, es torna a començar des del principi. En la segona fase, es modifica l'ordre dels caràcters. El missatge es divideix en blocs de quatre caràcters i les dues primeres posicions s'intercanvien amb les dues últimes. Per exemple, ABCD passaria a CDAB. Si l'últim bloc té menys de quatre caràcters, es deixa sense modificar. En la tercera fase, s'aplica una substitució utilitzant l'alfabet invertit. Això significa que A es converteix en Z, B en Y, C en X, i així successivament.
Com funciona la desencriptació?	Per desencriptar el missatge es realitzen les mateixes fases però en ordre invers. Primer es desfà la substitució amb l'alfabet invertit. Com que A correspon a Z i Z correspon a A, aplicar la mateixa substitució permet recuperar els caràcters anteriors. Després es desfà la transposició dels blocs de quatre caràcters. En el nostre cas, aplicar de nou el mateix canvi permet recuperar l'ordre original del bloc. Finalment, es desfà el desplaçament de la primera fase. En comptes de sumar el valor de la clau i la posició, es resten aquests valors. D'aquesta manera es recuperen les lletres originals del missatge.
Com sé que el meu sistema funciona?	El sistema funciona si, després d'encriptar un missatge i desencriptar-lo amb la mateixa clau,

	obtenim exactament el missatge original. També podem comprovar que, si utilitzem una clau diferent, el resultat de l'encriptació canvia i no podem recuperar correctament el text original.

3. Demostra el teu algoritme

Abans de començar a programar, feu un exemple manual complet.

Missatge: HOLA

Clau: Clave

Encriptació: H → ____ O → ____ L → ____ A → ____

Fase 1: Desplaçament amb la clau

Convertim les lletres a valors numèrics i sumem el valor de la lletra de la clau més la posició del caràcter.

H + C + 0 → J
O + L + 1 → A
L + A + 2 → N
A + V + 3 → Y

Resultat: JANY

Fase 2: Transposició

Intercanviem les dues primeres posicions amb les dues últimes.

JANY → NYJA

Fase 3: Substitució

Substituïm cada caràcter pel seu equivalent dins de l'alfabet invertit.

N → M
Y → B
J → Q
A → Z

Resultat: MBQZ

Igualment, desencripta el resultat i demostra que recuperes HOLA.

Desencriptació

Per recuperar el missatge original, fem les tres fases en ordre invers:

MBQZ ⟶ NYJA ⟶ JANY ⟶ HOLA

4. Implementació de la classe "Encripta" en Java

Crea una classe independent anomenada:

ClasseCriptografica

La classe haurà d'oferir, com a mínim, aquests dos mètodes:

- Encripta(Missatge, Clau)
- Desencripta(Missatge, Clau)

Els mètodes han de retornar el missatge resultant.

La classe criptogràfica no ha de contenir el programa principal. El programa principal serà qui utilitzarà la classe.

Programa principal ⟶ ClasseCriptografica ⟶ Encripta / Desencripta

5. Proves del sistema dissenyat d'encriptació

Test	**Prova**	**Resultat**
Test 1	Missatge curt: HOLA	X Correcte ☐ Incorrecte Observacions: El missatge s'encripta i es desencripta correctament.
Test 2	Missatge amb espais: HOLA MÓN	X Correcte ☐ Incorrecte Observacions: Els espais i els caràcters amb accent es conserven i es recupera el missatge original.
Test 3	Missatge més llarg: almenys 30 caràcters	X Correcte ☐ Incorrecte Observacions: El sistema funciona correctament també amb missatges llargs.
Test 4	Clau diferent: comprovar que el resultat canvia	X Correcte ☐ Incorrecte Observacions: En utilitzar una clau diferent, el missatge xifrat canvia.

Test 5	Encriptar i desencriptar: recuperar exactament el missatge original	X Correcte ☐ Incorrecte Observacions: Utilitzant la mateixa clau, es recupera exactament el missatge original.
Test 6	Clau incorrecta: explicar què passa	X Correcte ☐ Incorrecte Observacions: Si s'utilitza una clau incorrecta per desencriptar, no es recupera el missatge original.

6. Implementació d'un sistema de xifrat AES en Java

Fins ara heu dissenyat i implementat el vostre propi sistema de xifrat. Ara fareu un pas més: **utilitzareu AES (Advanced Encryption Standard),** un algoritme de xifrat simètric utilitzat en sistemes reals.

En aquesta part **no haureu de crear l'algoritme AES des de zero**. Java proporciona una API de criptografia que permet utilitzar-lo.

Objectiu

Crear una classe anomenada `ClasseAES` que permeti:

```
MISSATGE + CLAU
      ↓
     AES
      ↓
MISSATGE XIFRAT
```

i recuperar posteriorment el missatge original:

```
MISSATGE XIFRAT + CLAU
      ↓
     AES
      ↓
MISSATGE ORIGINAL
```

La classe haurà d'oferir, com a mínim, els dos mètodes següents:

```
public static String encripta(String missatge, String clau)
```

```
public static String desencripta(String missatgeXifrat, String clau)
```

6.1. Eines disponibles

Per realitzar aquesta activitat podeu utilitzar les classes i mètodes següents de Java:

Classes:

```
import javax.crypto.Cipher;

import javax.crypto.spec.SecretKeySpec;

import java.nio.charset.StandardCharsets;

import java.util.Base64;
```

Mètodes i elements que podeu consultar i utilitzar:

```
getBytes()

Cipher.getInstance()

cipher.init()

cipher.doFinal()

Base64.getEncoder()

Base64.getDecoder()

SecretKeySpec

Cipher.ENCRYPT_MODE

Cipher.DECRYPT_MODE
```

Aquestes són les eines que necessitareu per construir el vostre sistema.

No es proporcionarà el codi complet de la classe. Haureu d'investigar el funcionament de les classes i mètodes proporcionats i decidir com combinar-los per construir els processos d'encriptació i desencriptació

Trobareu un annex explicatiu breu al final del document.

6.2. Implementa la classe `ClasseAES`

Crea una classe independent anomenada ClasseAES, que tingui els mètodes:

encripta (missatge, clau)
desencripta (missatge, clau)

El mètode encripta() haurà de:

1. Rebre un missatge i una clau.
2. Preparar la clau perquè pugui ser utilitzada per AES.
3. Crear i configurar el sistema de xifrat.
4. Convertir el missatge al format necessari.
5. Xifrar el missatge.
6. Convertir el resultat en un format que es pugui retornar com a `String`.
7. Retornar el missatge xifrat.

El mètode desencripta() haurà de realitzar el procés invers:

1. Rebre el missatge xifrat i la clau.
2. Preparar la clau.
3. Crear i configurar el sistema de desxifrat.
4. Recuperar les dades xifrades.
5. Desxifrar-les.
6. Convertir el resultat novament a text.
7. Retornar el missatge original.

6.3. Programa principal

Crea un `ProgramaPrincipal` que utilitzi la classe `ClasseAES`.

Utilitza inicialment les dades següents:

```
Missatge: Aquest és un missatge secret.

Clau: 1234567890123456
```

El programa haurà de:

1. Mostrar el missatge original.
2. Encriptar-lo utilitzant AES.
3. Mostrar el missatge xifrat.
4. Desencriptar el resultat utilitzant la mateixa clau.
5. Mostrar el missatge recuperat.

El resultat final ha de demostrar que:

```
Desencripta(Encripta(missatge, clau), clau)   =    missatge
```

6.4. Experimenta amb el teu programa

Una vegada el programa funcioni, realitza les proves següents:

Prova 1. Clau correcta

Comprova que el missatge original es recupera correctament.

El missatge original es recupera

Prova 2. Clau diferent

Intenta desencriptar el missatge utilitzant una clau diferent.

Explica què passa.

El missatge original no es recupera correctament i el programa pot generar un error durant el procés de desxifrat. Per tant he afegit un trycatch en el main per capturar l'error.

Prova 3. Missatge diferent

Utilitza un altre missatge i comprova que el procés continua funcionant.

El sistema l'ha pogut encriptar i desencriptar correctament.

Prova 4. Clau de longitud incorrecta

Prova d'utilitzar una clau que no tingui la longitud adequada.
Observa què succeeix i explica el motiu.

Hem provat una clau amb una longitud no vàlida. El programa genera un error perquè AES necessita una clau amb una mida correcta.

6.5. Comprèn el codi

No n'hi ha prou amb aconseguir que el programa funcioni. Has de poder explicar què està fent.

Respon les preguntes següents:

1. Quina classe de Java s'encarrega de realitzar el xifrat?

La classe cypher és l'encarregada de realitzar el procés d'encriptació i desencriptació

2. Per a què serveix `SecretKeySpec`?

   Per convertir els bytes de la clau en una clau que pugui ser utilitzada per l'algoritme AES.

3. Què fa `Cipher.getInstance()`?

   Crea i configura un objecte cypher indicant quin algoritme de xifrat volem utilitzar. En aquest cas, AES.

4. Quina diferència hi ha entre `ENCRYPT_MODE` i `DECRYPT_MODE`?

   ENCRYPT_MODE indica que el Cipher s'utilitzarà per encriptar dades, mentre que DECRYPT_MODE indica que s'utilitzarà per desencriptar-les.

5. Per a què serveix `doFinal()`?

   doFinal() executa el procés de xifrat o desxifrat sobre les dades que li passem i retorna el resultat.

6. Per què hem de convertir el missatge a bytes?

   Perquè els algoritmes de xifrat treballen amb dades en format de bytes i no directament amb text.

7. Per què utilitzem Base64?

   Base64 serveix per convertir els bytes resultants del xifrat en una cadena de text que es pugui mostrar, guardar o transmetre fàcilment.

8. Quina mida té la clau que has utilitzat?

   La clau utilitzada té 16 caràcters, que equivalen a 16 bytes o 128 bits.

9. Què significa que AES sigui un sistema de xifrat simètric?

   Significa que s'utilitza la mateixa clau tant per encriptar com per desencriptar el missatge.

10. Per què necessitem la mateixa clau per recuperar el missatge?

    Perquè AES és un sistema de xifrat simètric. Si utilitzem una clau diferent, no podrem recuperar correctament les dades originals.

Finalment, respon:

Per què és preferible utilitzar un algoritme criptogràfic estàndard com AES en lloc de crear el nostre propi sistema de xifrat?

És preferible utilitzar AES perquè és un algoritme estàndard, àmpliament estudiat i utilitzat en sistemes reals. Un sistema creat per nosaltres pot contenir errors o vulnerabilitats que no coneixem, mentre que AES ha estat analitzat i provat durant molts anys.

Lliurament

Heu de lliurar en un repositori GitHub:

1. Sistema de xifrat propi

- `ClasseCriptografica.java`
- `ProgramaPrincipal.java`

2. Implementació AES

- `ClasseAES.java`
- `ProgramaPrincipalAES.java`

3. Documentació

- Document PDF amb les respostes i explicacions de la pràctica.

Cal lliurar aquesta documentació en un Repositori GitHub i que el compartiu amb la Cristina i en Carles.

Xifrat AES

El xifrat AES (Advanced Encryption Standard) va ser aprovat com estàndard de xifrat pel govern dels EUA després d'un procés de 5 anys. El xifrat va ser desenvolupat per dos criptòlegs belgues J.Daemen i V. Rijmen que el van enviar amb el nom de Rijndael.

Característiques:

- Xifrat de bloc: mida 128 bits.
- Clau de xifrat de 128,192 o 256 bits.

Actualment es considera un sistema de xifrat segur (la NSA exigeix clau de 192 o 256 bits per documents d'alta seguretat). És un algoritme ràpid, AES 256 és més ràpid que DES i per tant molt més ràpid que la solució 3DES.

Elements del xifrat:

- IV (Vector inicialització): És necessari per iniciar cada procés de xifrat. Té una mida de 16 bytes (1 bloc). El seu objectiu és evitar que dos blocs iguals tinguin el mateix criptograma.
- Clau (Key): És la principal informació per xifrar i desxifrar en els algoritmes simètrics. A la implementació per defecte de .NET s'utilitza la de 256 bytes.

Cipher mode:

ECB: xifra cada bloc per separat. Per tant, blocs idèntics donen el mateix criptograma.

CBC: s'utilitza un vector d'inicialització (IV) i cada bloc es combina amb el bloc anterior amb una operació XOR. Això assegura que blocs iguals no donin un criptograma idèntic.

Clau de xifrat:

Existeixen mètodes per obtenir la clau de forma automàtica, però això només té sentit si aquesta clau es pot enviar de forma segura. Volem obtenir la clau a partir de passphrasse que introdueixi l'usuari.

La forma més còmoda per l'usuari és introduir una password o passphrasse enlloc de una clau i un vector d'inicialització. Utilitzem Rfc2898DeriveBytes per obtenir un *hash* de la combinació dels dos textos. S'afegeix una cadena *salt* per fer més difícil l'atac per RainBow Tables. L'algoritme es pot aplicar iterativament per augmentar la complexitat.

Rúbrica:

Criteri	Què s'avalua	Punts	
1. Disseny del sistema propi	Defineix l'algoritme, la clau i explica correctament encriptació i desencriptació	**1,5**	
2. Implementació del sistema propi	ClasseCriptografica funciona correctament i està separada del programa principal	**1,5**	
3. Ús de la clau	La clau participa realment en el procés i una clau incorrecta no permet recuperar correctament el missatge	**1,0**	
4. Proves del sistema propi	Realitza i documenta correctament els tests proposats	**1,0**	
5. Implementació d'AES	Utilitza correctament l'API de Java i implementa ClasseAES amb encriptació i desencriptació	**2,0**	
6. Comprensió de l'API criptogràfica	Explica Cipher, SecretKeySpec, init(), doFinal(), Base64, etc.	**1,0**	
7. Experimentació amb AES	Realitza les proves amb diferents claus i missatges i explica els resultats	**0,75**	
8. Documentació i comparació	Respon les preguntes i compara el sistema propi amb AES de manera raonada	**0,75**	
9. Organització i lliurament	Estructura correcta del projecte, noms de classes, repositori GitHub i documentació	**0,5**	
TOTAL		**10**	